SPEAKER_NOTES

Speaker notes: Evaluating eFlx

CS637 video presentation. About 13.5 minutes of speech (~140 words per minute), roughly 15 minutes with slide changes and the demo clip.
Use these as a guide and say them in your own words.

Ghada Suleiman (slides 1–9, about 4.3 min)

Slide 1: Evaluating eFlx

Hello, we are Ghada, Rishikesh and Suraj, and we evaluate the ICCPS 2025 paper eFlx. No code was released, so we rebuilt the method from the paper alone, reproduced its experiments on real New York taxi data, critiqued it, and extended it with new ideas.

Slide 2: First the paper, then our work

First I summarise the paper: demand response, the problem, and the eFlx solution. Then Suraj and Rishikesh present our own work: our evaluation, implementation, reproduction, and new use cases and ideas. A navy badge marks the paper's content; orange marks our own work.

Slide 3: A taxi fleet as a power plant on wheels

An electric taxi uses more than 50 kWh a day, so a 13,000-taxi Manhattan fleet charges at around 100 megawatts. In an emergency, grids pay large customers to cut demand, often within 10 minutes; this is demand response. A factory has to stop working to do that. A taxi fleet doesn't: it has batteries, and a coordinator can shift when and where taxis charge without passengers noticing. A power plant on wheels.

Slide 4: A DR program has three stages

A DR program has three stages, the paper's Figure 1. In provision, the fleet has a contract but no call yet: it keeps serving passengers and may not charge extra, but it can get ready. In service, it cuts charging for two hours, the shaded area, and lives on stored energy. In recovery, it recharges. Because the call can come at any moment, staying ready is a continuous job.

Slide 5: Reacting helps; preparing helps more

The paper's motivating example, Figure 2. A fleet that only reacts when called already cuts 127.86 megawatt-hours over the two hours. A fleet that prepared in advance cuts 139.37, about

9% more, while serving the same passengers. So preparation, called provisioning, is worth something, and the paper asks how to do it best.

Slide 6: Be ready for a call that can come any time

The question: how should the fleet dispatch and charge taxis now, before any call, so that whenever a call comes it can cut as much charging as possible without losing passengers? Flexibility, Equation 6, is the charging with no call minus the charging when responding. It comes from spare taxis and stored battery energy. Three rules keep it fair: the call time is unknown, no extra charging before the call, and no worse service. The paper studies the hardest case, the evening peak.

Slide 7: The model: counting taxis, not tracking them

The model counts taxis instead of tracking each one: how many are vacant, occupied, charging or waiting, per region and battery level, in 20-minute slots. A driving taxi loses one battery level per slot; a charging taxi gains three. Each slot the operator decides where vacant taxis go and how many charge. All equations are linear, so every plan is a linear program that solves quickly.

Slide 8: Why it is hard: the yardstick moves

The hard part. Flexibility is baseline minus response, but the baseline, what the fleet does with no call, is the very policy being optimised. The yardstick moves with the thing it measures: an infinite-level problem. The paper first fixes the baseline to normal service, giving the bi-level problem in Figure 5. Solved exactly, that has about 1.5 million variables, far too slow for every 20 minutes. So it predicts normal service instead, and the two levels merge into one linear program, Problem 5.

Slide 9: eFlx: a closed-loop fleet controller

eFlx sits between the grid and the fleet, as in Figure 4: it receives DR signals and live taxi data, and sends back charging and dispatch orders. Algorithm 1 solves the linear program, re-predicts normal service, and repeats until the plan settles, usually in three or four rounds. Algorithm 2 runs this every 20 minutes and applies only the current slot’s orders. Importantly, eFlx always assumes the call may come in the very next slot. That is the paper’s method; Suraj now evaluates it.

Suraj Singh Raman (slides 10–17, about 4.1 min)

Slide 10: What the paper claims

To evaluate the paper, we start from what it claims. On about 13,000 Manhattan taxis, with the DR window at 17:20, eFlx charges 20% less during the window than normal operation, called

TRC, and offers 5.5% more flexibility. It comes within half a percent of an Oracle that knows the start time, at only about 2% overhead. These are the claims we set out to test.

Slide 11: Our critical reading: strengths and gaps

Next, our critical reading. The paper's strengths: an important new problem, an elegant escape from the infinite loop, real-time speed, and fair comparisons. But we found gaps, and for most of them we did something. Flexibility is measured against each policy's own baseline, so it can be gamed; Rishikesh shows this on Manhattan. eFlx always assumes DR comes next slot; our P-eFlx plans for when DR is likely. The service compromise is mentioned but never measured; we measure it. Only load cuts are studied; we test absorbing surplus power. Key parameters are missing, energy use is unrealistically high, trip probabilities are fixed, the Oracle isn't a true bound, and the Qv and Qo descriptions are swapped. To test the claims, we rebuilt the system.

Slide 12: Rebuilding eFlx with no code released

With no code released, the paper became our specification, one part per file. model.py holds the city's rules, Equations 1 to 4; policies.py the strategies, including eFlx and the baselines; sim.py the plan-act loop and the flexibility score. For Manhattan, fastplan.py builds the same linear programs about 20 times faster. Here is Equation 1 as code: every available taxi gets exactly one decision. Where the paper leaves numbers out, like battery size or charger counts, we chose values and documented each one in NOTES.md.

Slide 13: How we know our code is right

How do we know the code is right? The simulator checks every slot that no taxi is created or lost. And we have two independent builders of the same linear programs: a readable one that mirrors the paper's equations, and the fast one we use at scale. On four test cases they reach exactly the same optimal values.

Slide 14: Toy model: the core idea reproduces

First, a toy city small enough to check by hand: 2 regions and 100 taxis. The core idea reproduces: eFlx raises flexibility from 870 to about 1270 kWh and cuts charging during DR by 38%, serving every passenger, close to the Oracle. Notice the jump back to full charging at 19:20; Rishikesh returns to that rebound.

Slide 15: Same optimum, different answers

A reproducibility finding. Our two builders always reach the same optimal value, but often pick different optimal solutions, because these programs have many ties. Over a full run that changes results by up to 14%: the Oracle charges 650 or 561 kWh depending on the builder.

So a gap like “0.52% from the Oracle” can sit inside solver noise; papers should state their tie-breaking. Next, a quick demo.

Slide 16: Demo: anyone can rerun it

over the screen recording Here is our repository; the README explains how to run everything. The validation test takes four seconds, and the toy experiment seven, printing the table you just saw and regenerating its figure. The Manhattan runs use the same code but take hours, so we show their saved results.

Slide 17: Real data: NYC taxi trips, March 2024

For the real experiment we used New York’s taxi trip records for March 2024: about two million weekday trips in Manhattan, about 2,400 per 20-minute slot at the evening peak. The paper’s 38 grid regions aren’t public, so we clustered taxi zones into 12 regions. We kept the paper’s battery and timing values and chose the rest, such as the number of chargers. Rishikesh now presents the results.

Rishikesh Sawant (slides 18–25, about 4.2 min)

Slide 18: Figs. 6 and 7: paper vs ours (3,200 taxis)

The paper’s Figures 6 and 7 beside ours: flexibility on top, DR charging below. A first surprise: at the paper’s own fleet size of 13,000 taxis, every policy cuts DR charging to zero, so nothing can be compared. So we use a tighter fleet of 3,200 taxis. There the ordering holds in direction, eFlx charging least, then R2I, then normal operation, but the error bars across days are large.

Slide 19: Claim by claim: direction holds, size doesn’t

Claim by claim, totals over three weekdays. eFlx cuts DR charging by 4.8%, not 20%, and ends 4.6% from the Oracle, not half a percent. Flexibility, plus 4% against 5.5%, is the closest match. Without the charging limit, eFlx is actually worse than R2I and TES. Service stays the same, but nearly all the gain comes from one of the three days. The likely causes: unpublished parameters, the fleet size, and solver ties.

Slide 20: Overheads, and what the experiments miss

Overheads are higher than claimed: 10% more idle driving and 44% more waiting, against about 2%. Runtime matches, about four minutes per slot. Look at the right edge: after 19:20 charging shoots up to 26 megawatts, a new peak for the grid that the paper never measures. And at 13,000 taxis eFlx scores 40% more flexibility while every policy charges zero: the metric problem from slide 11, in action.

Slide 21: Fig. 11 and Table 1: lead time and forecast error

Two more paper experiments. Figure 11 varies how early provisioning starts. On the toy city, longer preparation steadily helps. On Manhattan it only partly holds: no gain at 20 to 40 minutes, a peak at 80, then a dip. Table 1 adds a 10% error to the demand forecast. The paper reports flexibility down 1.3%; we see 6.8%, with about 5% of passengers missed, which the paper doesn't report. And eFlx's edge over normal operation almost vanishes: 2.67 against 2.63 megawatt-hours.

Slide 22: New use cases: when does eFlx help?

Now new use cases. The most revealing is a fleet-size sweep: what is the least charging possible during DR while serving everyone? From 13,000 taxis down to 4,000 the answer is zero; there are so many spare, charged taxis that any policy can stop charging. Only at 3,200 does some charging remain. So the paper's fleet sits in the easy corner of its own problem. We also varied the DR start time and used several weekdays.

Slide 23: Absorbing surplus solar and wind power

A second use case. Grids with lots of solar and wind sometimes have too much power and want fleets to charge more. We reversed the response to maximise charging instead. On Manhattan the fleet can absorb only 0.1 megawatt-hours extra, against 2.6 to 2.9 of cut, because normal operation already uses every free charger. To offer both directions, a fleet must hold some charging back in advance: a two-sided plan the paper doesn't consider.

Slide 24: P-eFlx: plan for when DR is likely

Our first new idea, P-eFlx. eFlx always assumes DR comes in the next slot, which is pessimistic; the Oracle knows exactly when, which is unrealistic. P-eFlx uses a probability distribution over start times and minimises the expected DR charging, with the same problem size. It cuts DR charging by 32% versus eFlx on the toy city and by 2.6% on Manhattan, closing three quarters of the gap to the Oracle without knowing the start time.

Slide 25: How much flexibility does a little service buy?

Our second idea measures the service compromise the paper mentions but never quantifies: during DR, the fleet may serve slightly fewer passengers. On Manhattan, missing about 4% of passengers gives two and a half times the flexibility, from 2.86 to 7.4 megawatt-hours, while eFlx's provisioning is worth less than 1% of service. This curve is exactly what a grid operator and a taxi company would need to price a DR contract.

All three (slides 26–27, about 0.9 min)

Slide 26: What the paper did, and what we did

Ghada: The paper defined fleet flexibility and turned a hard bi-level problem into one fast linear program, claiming 20% less charging in an emergency.

Suraj: We rebuilt it from scratch and validated it; our limits are chosen parameters, 12 regions instead of 38, and a fluid simulator.

Rishikesh: On real data the idea holds, but the gain shrinks to about 5%. We added P-eFlx, the service trade-off and the surplus-power test; next would be a per-taxi simulator and two-sided plans.

Slide 27: Contribution

Ghada: Thank you for watching our evaluation of eFlx.

Suraj: Everything you saw today, the code, the data, the results and the full report, is in our GitHub repository, so you can rerun any experiment yourself.

Rishikesh: We look forward to your questions. Thank you!

Total: 1887 words, about 13.5 minutes of speech.